

University of Padova

Department of Mathematics

Master Thesis in Computer Science

Self Assessment Tool for Medical Exam Candidates

Supervisor

Professor Massimo Marchiori

University of Padova

Co-supervisor

Davide Baldelli

Loop AI

Master Candidate

Alireza Dehghan

Academic Year

2023-2024

Graduation Session: 23, September 2024

Abstract

Students' knowledge assessment, particularly by oral examinations, is a resource-consuming process classically requiring human experts. This thesis investigates the development of a self-assessment tool aiming at making possible the automated evaluation of medical exam candidates. Powered by current developments in NLP and multimodal analysis, this will help with the automation of question generation and the evaluation of student responses. The tool has two main components: the natural language processing and the multimodal analysis components. The NLP component is responsible for question generation from provided textbooks, speech-to-text for oral responses, and further evaluation of such responses. This is achieved through the use of advanced language models (LLMs) in ensuring parsing and generation of accurate medical-related content. This evaluation is enriched with multimodal analysis that will treat prosodic features and facial expressions to identify student confidence and uncertainty to resemble an evaluation as of human examiners. Alongside the system, there is a web application developed for users to upload educational materials and self-assess themselves, providing grounds for testing and experiments. The conducted experiments have shown that the system provides effective and accurate feedback in an acceptable duration. This self-assessment tool is a notable step toward progress in medical education by providing a scalable, efficient, and accurate way of evaluating students, with the ultimate goals of improving learning outcomes while easing educators' workload.

Contents

Abstract iii

List of figures vii

List of tables ix

1 Introduction 1

2 Background 3
- 2.1 Natural Language Processing (NLP) . . . 3
- 2.2 Automated Question Generation (AQG) . . . 5
- 2.3 Automated Exam Evaluation . . . 7
- 2.4 Multimodal Analysis of certainty . . . 8

3 Methodology 11
- 3.1 Exam Creation Concept . . . 11
- 3.2 Natural Language Component . . . 12
  - 3.2.1 Question Generation . . . 12
  - 3.2.2 Speech to text . . . 13
  - 3.2.3 Evaluation System . . . 14
- 3.3 Multimodal Analysis Component . . . 17
  - 3.3.1 Prosodic based evaluation . . . 17
  - 3.3.2 Facial features based evaluation . . . 20

4 Implementation Setup 23
- 4.1 Tools Used . . . 23
- 4.2 Internal Modules . . . 24
  - 4.2.1 Input Handling . . . 25
  - 4.2.2 Question Generation . . . 26
  - 4.2.3 Answer Assessment . . . 27
  - 4.2.4 Visual Certainty Assessment . . . 27
  - 4.2.5 Auditory Certainty Assessment . . . 28
- 4.3 Integration (UI) . . . 28

5 Discussion 33
- 5.1 Testing Methodology . . . 33

5.2 Results and Comparisons 36
5.3 Challenges and Limitations 38

6 Conclusions and Future Work **39**

References **41**

Acknowledgments **51**

Listing of figures

3.1 The conceptual flow of the program. 12
3.2 An example of the prompt used for question generation. 13
3.3 A depiction of the preprocessing pipeline. 14

4.1 The first page of the application. 28
4.2 The application synthesising the uploaded PDF. 29
4.3 Application in review mode, the user has chosen to keep this section 30
4.4 A student ready to take the exam inside application 30
4.5 A student is taking the exam inside application 31
4.6 A student has finished the exam and has sent their responses 31
4.7 The scores of a taken text emailed to the student 32

Listing of tables

3.1 Comparison of Similarity measures between preprocessed and untouched sentences 16
3.2 Feature Analysis of Whole Utterance 18
3.3 Performance metrics for the random forest based classifier. 19
3.4 Performance metrics for the XGboost based classifier. 20
3.5 Performance metrics for the Neural Network based classifier. 20
3.6 Percentage of Items with Specific Visual Cues for High and Low Confidence Conditions 21
3.7 Probability of Low Confidence Given a Specific Visual Cue 21

5.1 A sample of the evaluated questions by the expert. 36
5.2 Statistics from the evaluations done by the expert, 1 is considered bad in quality and 5 means high quality 37
5.3 Performance metrics of our system's evaluation with regards to the expert's, along with 95% confidence intervals calculated using Clopper-Pearson interval. 37
5.4 Deviation Metrics for Semantic similarity and Entities similarity. 38

1 Introduction

Assessing students' knowledge on a specific subject is a tedious task that requires concentration and time—both of which are scarce resources, as an expert in the subject needs to evaluate dozens of students at once. Each conventional academic evaluation consists of two stages: the first involves creating questions related to the material being discussed, and the second entails assessing the answers, whether written or oral, to verify the validity of the knowledge gained by the students. Although evaluation demands time and energy from both students and professors, it significantly helps students solidify their learning and is one of the most important components of education in general.

Given the positive impact of these evaluations on students, we have seen the inclusion of self-assessment quizzes in many educational materials. These quizzes consist of multiple-choice or open-ended questions designed to reflect what the student should have learned from previous topics. However, these quizzes often provide no feedback or evaluation, leaving students with nothing to improve upon. Additionally, since the number of questions created is finite, there may arise a need for more questions, which is a time-consuming task requiring expertise that may not always be available.

Medical educational materials are particularly well-suited for automating the process of self-assessment, as patients' medical diagnoses and treatments are based on case-specific protocols that denote a standard set of instructions or items. This characteristic underscores the fact that for a successful evaluation, a student must simply recite a repeatable and standard list of items and steps, which are medical-related terms and concepts, thus allowing for more robust

algorithmic fact-checking with higher accuracy.

To create a robust system for this task, our objective is to develop a system that generates questions from a given subject and assesses the students' answers, grading them and evaluating whether they are ready or need more practice. The goal is for this system to resemble an evaluation done by an expert as closely as possible, taking into account the students' stress levels and certainty in their answers. Such a system has been made possible by recent advancements in AI, particularly in the field of Natural Language Processing (NLP). The development of Large Language Models (LLMs) has created an ideal environment for building an educational tool like this.

In addition to the power of LLMs, another technology that has revolutionized NLP and enables us to perform this task accurately and efficiently are language models such as BERT (Bidirectional Encoder Representations from Transformers) and RoBERTa (Robustly Optimized BERT Approach). These language models allow us to comprehend complex human-generated text. Together with LLMs, they are the perfect tools to generate relevant, meaningful questions and assess students' responses with high accuracy.

Speech-to-text technologies have also improved significantly, enabling us to use recorded voice as input with almost no data loss in the process of converting speech to text. This provides an experience as close as possible to a real examination by a human expert while supplying precise feedback to students.

This thesis is structured as follows: First, the previous works are reviewed in Chapter 2. Then, we discuss our methodology and theoretical implications in Chapter 3, covering the conceptual pipeline and components of the system. In Chapter 4, our implementation setup is reviewed, detailing the tools used to deploy the system and highlighting the design choices made. Chapter 5 presents the testing procedures conducted, discusses the results, and compares them with previous works in the field. Concluding remarks are provided in Chapter 6.

2 Background

In this chapter, we provide the foundational background necessary to understand the development and functionality of the system. We begin by discussing the underlying concepts and theoretical frameworks that inform the design and implementation of our self-assessment tool. In Section 2.2, we review the current landscape of automated evaluation systems, highlighting their strengths and limitations. This sets the stage for understanding the innovations introduced in our system. Section 2.1 explores advancements in Natural Language Processing (NLP) and multimodal analysis, providing context for their application in question generation and response evaluation. Finally, Section 2.3 covers the specific challenges and requirements of evaluating medical exam candidates, illustrating the unique needs addressed by our approach and the relevance of integrating NLP with multimodal techniques. This background establishes a comprehensive foundation for the detailed discussion of our system's components in the subsequent chapters.

2.1 Natural Language Processing (NLP)

NLP models manipulate textual data for further analysis and have evolved significantly with the progress in machine learning and deep learning fields. NLP models have seen a rise in both popularity and complexity following the generational leap in the processing power of the devices and the advancements in AI. The NLP models have transitioned from simple statistical methods of analysis to state-of-the-art, computation-heavy language models[1]. A very

recent breakthrough in the field of AI which overhauled the NLP world was the introduction of Transformer architecture[2] which was intended to be an experiment for the task of machine translation, where they came on top of the previously used Convolutional Neural networks.

Following the introduction of the Transformer, a model based on the same architecture, named BERT[3], was introduced. BERT performs exceptionally well on a wide range of sentence-level and token-level tasks. Building on the idea of pretrained transformers, OpenAI's GPT (Generative Pretrained Transformers)[4] models were developed. Unlike BERT, which is an encoder-only model, GPT can generate human-like textual content. Another generative effort which was built upon the transformers architucture is the T5 model[5]. The key idea behind T5 is to treat every NLP problem as a text-to-text problem, where both inputs and outputs are text strings. This approach allows T5 to handle a wide variety of tasks, such as translation, summarizing, question answering, and more, using a single model architecture.

Touvron et al. focused on training a series of language models, known as LLaMA[6], with the goal of achieving the best possible performance across different inference budgets. To accomplish this, the models are trained on a larger number of tokens than is typically used in other models. The LLaMA models range in size from 7 billion to 65 billion parameters and demonstrate competitive performance when compared to the best existing LLMs. An updated version of LLaMA, named LLaMA 2[7], was made available in 2023. LLaMA 2 differs from LLaMA 1 by being trained on a new mix of publicly available data, with a 40% larger pretraining corpus. It also features a doubled context length and incorporates grouped-query attention. The latest advancement, LLaMA 3[8], introduces a new set of foundation models that significantly extend the capabilities of its predecessors. With the largest model having 405 billion parameters and a context window of up to 128K tokens, LLaMA 3 supports better multilinguality, coding, reasoning, and tool usage. It also includes integrated image, video, and speech capabilities.

There are multiple efforts on literature to bring the power of encoder models such as BERT and generative models such as T5 or GPTs to the medical domain, one such work is medical mT5[9], the first open-source text-to-text multilingual model for medical applications and human-AI interaction in the medical field with great success. Goel et al. show how LLMs can reduce the difficulty and human intervention of medical information extraction task from clinical notes within electronic health records[10]. They also impressively utilize prompt engineering to achieve their results with no fine tuning, which we will discuss later. Truechin et al. evaluated the performance of BERT based models in identifying complex medical concepts from narrative medical documents including discussions on bariatric surgery, statin treatment

non-acceptance, and tobacco use status. Med-PaLM[11] and Med-PaLM 2[12] are efforts towards better question answering in medical section by domain-specific fine-tuning, and novel prompting strategies of a base LLM.

As seen in efforts like EvalQuiz [13], LLMs have become essential NLP tools for generating quality quizzes for self-learning, and they are also our method of choice. LLMs encode input text into a high-dimensional vector space, maintaining semantic relationships between words and phrases. The prompt used as the input for these LLMs can significantly affect the quality of the results [14]. Chen et al. have shown how even the simplest forms of prompts can improve a model's output, with more complex forms of prompting, such as one-shot and few-shot prompting (providing one or a few examples of the desired output as input), substantially enhancing the quality of the results [15]. Knoth et al. have examined the usage of AI by non-expert students and demonstrated how "AI literacy" and high quality prompting is crucial for obtaining desirable results from generative models[16]. Heston et al. have elaborated on the importance of prompt engineering in medical sector when using generative language models, pointing out the problems sub-optimal prompts might create such as bad citations, hallucination and unmeaningful answers.[17]

Given NLP's wide range of applications, there are task-specific models that may perform better than others. spaCy[18] is a library for advanced NLP that uses neural networks for tasks such as tagging, parsing, named entity recognition (NER), text classification, and more. scispaCy[19] is an effort to develop a domain-specific NLP library by retraining spaCy's models for POS tagging, dependency parsing, and NER on biomedical text, while also enhancing the tokenization module by incorporating additional rules beyond the original. Our project extensively uses scispaCy for the NER task.

2.2 Automated Question Generation (AQG)

AQG refers to the use of computational techniques to create questions from textual content automatically. This process leverages NLP techniques to generate diverse and contextually relevant questions.

The search for Automated Question Generation dates back to 1976, when Wolfe designed a system named AutoQuest [20]. It heavily utilized pattern matching and part-of-speech recognition to form simple questions. Given the limitations of NLP at that time, it was substantially restricted to simple, non-abstract texts.

With various methods emerging in subsequent years, researchers have historically catego-

rized the question generation approaches into three main techniques[21][22]:

- Syntax-based approaches rely on manipulating a sentence's syntax tree. A typical question generation process consists of pipelined operations such as unmovable phrase marking, question phrase insertion, main verb decomposition, subject auxiliary inversion, and question formation.
- Template-based techniques use domain-specific templates for question generation, which are generated and chosen by human experts based on answer phrases. Since creating templates for generic topics is challenging, this approach is best suited for special-purpose applications.
- Semantics-based approaches generate logical and deep learning questions (DLQs). The Semantic Role Labelling (SRL) process identifies the semantic arguments associated with a sentence's verb and assigns them their specific roles. The semantic role labeller identifies mandatory arguments such as "subject," "object," and "indirect object," as well as optional arguments, which can be used to construct questions.
- ontology-based methods, in which the system creates questions by incorporating external information, replacing words in input sentences with concepts from a domain-specific ontology knowledge base.[23]

While most of the literature prior to 2021 utilized only one of these approaches in their AQG systems[24], there are efforts that have combined multiple approaches to create a wider variety of questions. Notable among these is PolyAQG[25]. Blšták et al. introduced a novel approach to question generation by implementing traditional methods, namely Semantics and Template-based approaches, and incorporating machine learning to learn new transformation rules [26]. The advancements in machine learning methods for question generation are largely attributed to the creation of the SQuAD dataset[27], originally intended for reading comprehension, which has also benefited the AQG task.

Recent advances in the adoption of LLMs in education have opened up new possibilities for automating various educational tasks, including the generation of self-assessment quizzes. Olney[28] investigates the effectiveness of multiple LLMs in generating multiple choice questions from a college science textbook. According to the study, finetuned Macaw and instruction-tuned Bing Chat models perform similarly to human authors on six of seven evaluation criteria. However, each model exhibits distinct error patterns: Macaw frequently generates repetitive answer options, while Bing Chat occasionally fails to include the correct answer among the choices. Despite these drawbacks, Macaw's performance aligns with human standards when error items are excluded, demonstrating that LLMs hold substantial potential for automating

educational content generation with further enhancement. This study highlights the potential of LLMs to reduce the cost and time involved in MCQ creation, provided their outputs are regularly monitored for specific error tendencies. Doc2Query[29] is a successful method for document expansion used in tasks like document retrieval. The approach involves training a transformer-based model to generate relevant questions or queries from an input document. These generated queries are then appended to the original document to enhance its indexing and improve retrieval performance. DocT5Query[30] is the successor of the authors previous work which uses the T5 language model to obtain the queries and at the time had state of the art performance in MS Marco passage ranking.[31]

EvalQuiz[13] explores the feasibility of using OpenAI's GPT-4 to automatically create self-assessment quizzes based on lecture material. The authors designed a pipeline that simplifies the quiz development process by allowing instructors to input lecture materials, which are then processed through topic modeling, keyword extraction, and prompt engineering to generate multiple-choice questions (MCQs). While the automated approach significantly reduces quiz creation time and aligns with the lecture topics, the study found that the generated questions lacked originality and versatility. EvalQuiz has adopted a similar approach to ours in addressing the inherent challenges of question generation, with the exception of focusing on multiple-choice questions and operating within a less sensitive context, specifically Software Engineering education.

2.3 Automated Exam Evaluation

Automated exam evaluation involves leveraging computational tools to assess and grade exam responses efficiently, minimizing the need for human involvement. This approach typically employs NLP techniques to analyze text, comparing answers to reference responses and evaluating factors such as content relevance, keyword usage, and grammatical correctness.

Patil et al. developed a system to evaluate descriptive answers, extensively utilizing NLP applications such as POS tagging, tokenizers, keyword extraction, and grammatical error checking.[32] Their system operates primarily on a points system, assigning points by comparing keywords against a database of synonyms and antonyms, and by checking for grammatical errors and word positioning. Raut et al. followed a similar approach, creating a system that assesses the similarity between a given answer and a reference answer by analyzing keywords and synonyms[33]. They also incorporate a well-formed reference file from a human expert to identify which synonyms should be considered and what incorrect words to flag to determine if an answer is in-

correct.

Rahman et al. introduced a method for the evaluation of answer scripts that first used OCR techniques to extract text from images[34]. After preprocessing the text, four similarity measures—synonym similarity, bigram similarity, cosine similarity, and Jaccard similarity—were computed as parameters to obtain a final grade. To achieve acceptable results, they conducted a survey and manually assigned weights to each similarity measure to ensure comparable grading with human experts. Agarwal et al. introduced a text evaluation system named AutoEval[35] which takes an NLP approach to solve the exam evaluation task. They primarily used a bag-of-words model to convert text into numerical representations, enabling the calculation of similarity scores between the student's responses and the correct answers stored in a database. Building on previous research, Bashir et al. proposed a novel methodology based on machine learning and NLP techniques. Their paper aims to develop a more robust system for evaluating descriptive answers by incorporating various methods such as WordNet, Word2Vec, word mover's distance (WMD), cosine similarity, multinomial naive Bayes (MNB), and term frequency-inverse document frequency (TF-IDF).[36] The proposed system uses solution statements and relevant keywords to train a machine learning model capable of predicting grades more precisely.

2.4 Multimodal Analysis of certainty

The literature has historically divided the sources of uncertainty into two categories. *Internal uncertainty* is the doubt or lack of confidence that originates within an individual. It is often related to the individual's own knowledge, feelings, or cognitive processes. *External uncertainty* comes from factors outside the individual and typically relates to the environment or external conditions [37]. In our research, we only consider internal uncertainty, which, in the context of exams, should stem from a lack of knowledge or confidence in the given answer.

There have been only a few studies on measuring certainty by utilizing multimodal analysis. Dijkstra et al. took a novel approach to examine whether multimodal cues, including prosodic features such as fillers, rising intonation, and facial expressions, impact the perception of a speaker's confidence while answering factual questions [38]. The researchers found that all examined cues were correlated with the perception of confidence, with fillers being perceived as a sign of confidence rather than uncertainty. Another major finding from this study was that these cues operate independently.

Pon-Barry examines only the acoustic and prosodic features to investigate the correlation be-

tween those features and uncertainty.[39] The researchers collected speech samples from participants with varying degrees of certainty and performed perceptual and statistical analyses on the data. The primary focus was on identifying prosodic features such as pitch, energy, and timing that indicate a speaker's degree of certainty. The analyses revealed that these prosodic features appear not only in the specific words or phrases that cause uncertainty, but also in the surrounding context. This implies that the expression of confidence or uncertainty in speech is spread across a broader temporal span, rather than being limited to a specific moment of uncertainty.

Mori et al. demonstrated that observers could accurately assess a speaker's confidence based on the speaker's own subjective confidence levels. Observers distinguished between low and high confidence levels by focusing on the speaker's eyes, facial expressions, and head movements. The study found that facial cues, particularly those in the eye region, play a significant role in how observers implicitly decode a speaker's moment-to-moment confidence. However, the reliance on specific visual cues varied between speakers, suggesting that individual differences may influence how confidence is visually communicated and perceived [40]. Additionally, Emerson et al. recently attempted to predict user confidence in video recordings by extracting multimodal features, such as eye gaze and prosodic features, and conducting elimination experiments to determine their predictive power. The results showed significant improvements in confidence prediction when both visual and verbal features were used [41].

3

Methodology

In this chapter, we delve into the theoretical logic behind the system and its components, describing how each module functions. First, in Section 3.1, we demonstrate the pipeline and review its main elements. Then, in Section 3.2, the Natural Language Component is broken down into its smaller segments, explaining how the question generation, answer evaluation, and speech-to-text processes work. Finally, we provide detailed insights into the multimodal analysis component in Section 3.3, showing how prosodic and facial feature-based evaluations are combined to detect uncertainty in the user's responses.

3.1 Exam Creation Concept

Logically, to reach our goal of question creation and the evaluation of answers and certainty levels, our system needs to have two main components: a natural language component and a multimodal analysis component, as seen in the figure3.1. Our pipeline begins when an input is received. The NLP component starts processing the file and generating questions, while the student attempts to answer them in almost real-time, recording themselves and uploading the responses for evaluation. After a successful exam session, the natural language and multimodal analysis components work in parallel to assess the correctness and certainty of each submitted answer, and then send feedback back to the student.

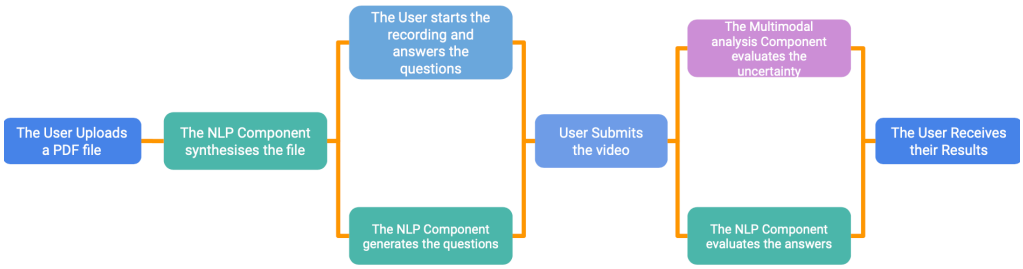


Figure 3.1: The conceptual flow of the program.

3.2 Natural Language Component

The natural language component is possibly the most important part of our system, bearing the majority of the responsibilities. It consists of three main subsystems, each dedicated to a specific task. The question generation pipeline, explained in Section 3.2.1, not only generates questions but also selects the best question to ask the student. In Section 3.2.2, we cover the speech-to-text pipeline and the post-processing applied to the input once it is converted to textual format to aid in evaluation. Finally, we provide details on the evaluation pipeline, explaining our evaluation criteria and the tools used to measure the correctness of a response.

3.2.1 Question Generation

Our question generation pipeline begins by parsing the input, which is in the form of a PDF file, and dividing it into smaller sections. Since PDFs do not contain metadata indicating the start and end of chapters or sections, we perform this division statistically by analyzing multiple cues, including word usage, font sizes, font types, and sentence lengths. After successfully dividing the text, the system takes each section and creates smaller, overlapping chunks of text with arbitrary lengths and an arbitrary number of overlapping words. This approach is beneficial because we aim to create a modular system independent of the LLM used, as different LLMs have varying capabilities regarding the number of words they can process as input. Each chunk of text is then embedded into a prompt designed to generate a question along with possible answers derived from the text. An example of this prompt can be seen in Figure 3.2.

As seen in the prompt, each generation should produce a question, two correct answers, a paraphrased answer and a wrong answer. The need for the paraphrased and wrong answers arises from our question selection criteria, where we assign a penalty score to each rank using the equation 3.1.

```
"""
[INST]
You are a medical Quiz creator, with no prior knowledge. if there are abbreviations in your answers,
mention the full word inside a parenthesis exactly after it. then I want you to follow all the steps
below carefully.

#Step 1: if the text is not good enough to create questions from, write "no questions can be
generated" and end

#Step 2: Strictly and only from the text, generate a question about the details or steps of a medical
procedure explained in the text. write the question in front of GenQues:

#Step 3: strictly using the text, answer the question created in step 1, the answer should be
complete and concise, answering the question in detail and step by step. write the answer in front of
GenAns:

#Step 4: the answer in step 3 might contain additional information that is not related to the
question from step 2, only give the answer without unnecessary information. write the answer in front
of PrfctAns:

#Step 5: if the text is not good enough for answering the question, write "no questions can be
generated"

#Step 6: Based strictly on the answer from step 3, write a new text which covers exactly all the
points mentioned. Do not change the key topics. Use simpler terms to describe the content. write the
answer in front of GdAns:

#Step 7: Generate a completely false answer using your own knowledge for the question in step 2.
write the false answer in front of IncorrectAnswer:

text: ###
Obesity and arterial hypertension commonly occur in the same patients and often have type 2 diabet...
###

[/INST]
"""
```

Figure 3.2: An example of the prompt used for question generation.

$$PenaltyScore_i = (100 - SP_i * 100)^2 + (SW_i * 100)^2 \tag{3.1}$$

In the formula, SP denotes the cosine similarity between the BERT encodings of the paraphrased answer and the correct answer, which is a number between 0 and 1. Similarly, SW represents the cosine similarity between the BERT encodings of the wrong answer obtained from the LLM and the correct answer. The penalty score essentially penalizes the distance of similarity of the paraphrased answer from being 100% correct and the wrong answer from being 0% correct. This criterion ensures that there is sufficient dissimilarity between an arbitrary wrong answer and a valid answer, which contributes to more accurate evaluation later on. After calculating the score for each set of question-answer pairs, they are sorted in ascending order, and then the top N questions are selected for the self-assessment, where N is an arbitrary number.

3.2.2 Speech to text

After a successful examination, the received video file must be divided into several smaller parts, each containing the response to a single question. Each part is then given as input to a pre-

trained speech-to-text model. The resulting raw text files are used as input for a post-processing pipeline, in which filler words are removed. These filler words include terms like "ummm," "hmmm," "thing," "like," and so on.

After removing the filler words, numerical numbers are converted to words, and abbreviated words are expanded to their full versions. We use a comprehensive list of technical medical abbreviations obtained from South Eastern Sydney Local Health District [42] to accurately address them. Since addressing abbreviations is essential for accurate evaluation and new abbreviations may be introduced over time, the system also considers possible abbreviations from all keywords in the answer by examining the first word of each detected keyword longer than two words.

The post-processed text is then ready to be used in the evaluation after the embeddings are completed.

3.2.3 Evaluation System

The evaluation system consists of a pipeline for preprocessing the responses and the answers in the database, followed by another pipeline with multiple evaluation metrics to verify the correctness of a given response.

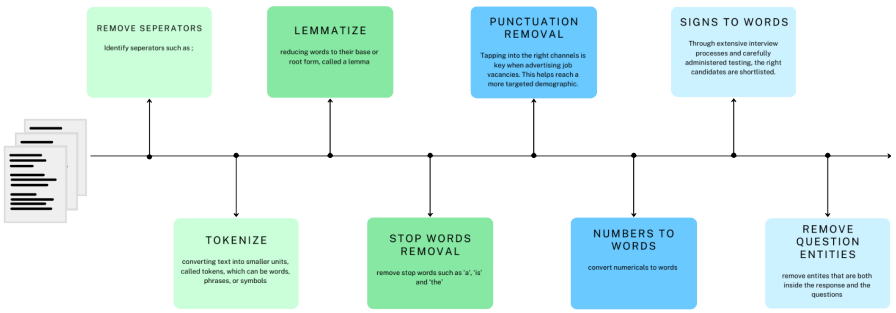


Figure 3.3: A depiction of the preprocessing pipeline.

The preprocessing pipeline, shown in figure3.3, starts with isolating the numbers used in the string with extra spaces to avoid connecting words with numbers. Next, all separators (e.g., '-',

';', ':', '|', '_' etc.) are replaced with spaces since they do not hold significance in our metrics but may introduce unnecessary complexity. The resulting string is then tokenized and lemmatized to ensure there are no differences between words from the same lemma, e.g., no distinction between "Talks" and "Talk". After lemmatizing the tokens, stop words are removed. Stop words are common words such as 'the', 'is', 'in', 'and' etc. that usually do not carry significant meaning or contribute to the overall semantics. Removing these words helps focus on the more meaningful components of the text and reduces noise in the data. Since stop words are typically not essential to the content of the sentences, they are filtered out to simplify the analysis and improve the efficiency of the subsequent processing steps. Finally, punctuation is removed to reduce the complexity of the sentences and facilitate easier comparison of similar sentences.

The numerics are converted to lexical numbers to standardize the numbers used across different responses. In the next section of our pipeline, symbolic representations of inequalities and other relational operators are converted into their corresponding lexical forms. Specifically, the greater than sign ('>') is replaced with the phrase 'more than', and the less than sign ('<') is substituted with 'less than.' Similarly, other operators are handled: the greater than or equal to sign ('≥') is converted to 'greater than or equal to', the less than or equal to sign ('≤') is translated to 'less than or equal to', and the equal sign ('=') is replaced with 'equal to.' This approach ensures that the text is standardized and matches the expected output from our speech-to-text pipeline. This is particularly useful in contexts where verbal explanations of mathematical concepts need to be verified by the similarity metric using the encoded representations of the words.

Given our testing, we have shown that removing entities that appear in the question from the responses greatly increases our accuracy in evaluation. Therefore, using a pre-trained model, we conduct entity recognition on the text of the question and delete those entities from the responses. This is because entities that appear in the question often also appear in the response to create a comprehensive answer, which increases the similarity of the responses with the correct answers, even when the responses are not valid but contain the themes and keywords from the question. In Table 3.1, we can see the average similarities of original sentences and the average similarities of sentences with question entities removed. It is clear that removing question entities considerably decreases the similarity between the correct answer and the wrong answer, while a paraphrased answer only shows a marginal change.

After the preprocessing pipeline, our correctness verification pipeline can commence. To measure correctness, we compare the derived entities from the student response and the answers, then attempt to partially match the words with an arbitrary confidence to minimize the effect of incorrectly detected words in our speech-to-text pipeline. Using the definitions of

Measures	Type	Untouched	Preprocessed
Avg. Cosine Similarity	Paraphrased Answer	0.89	0.84
Avg. Cosine Similarity	Wrong Answer	0.69	0.47
Avg. Entities Similarity	Paraphrased Answer	0.67	0.68
Avg. Entities Similarity	Wrong Answer	0.25	0.16

Table 3.1: Comparison of Similarity measures between preprocessed and untouched sentences

recall 3.2 and precision 3.3, we compute the F1 score utilizing the equation 3.4.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{3.2}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{3.3}$$

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3.4}$$

The lowest score is 0 and the highest is 1, which indicates perfect matching answers. We use the F1 score to strike a balance between precision and recall.

To verify correctness, in addition to the used entities, it is necessary to ensure the semantics in which those entities are used. The semantic similarity of the response with respect to the answers is not measurable in the lexical format; therefore, it is essential to convert the texts into numerical representations. This is achieved by using a language model to encode the sentences. With the encodings, the semantic similarity can be calculated by computing the cosine similarity between the encodings of the student response and the actual response, resulting in a value between 0 and 1.

$$\text{Cosine Similarity} = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} \quad \begin{cases} \mathbf{A} \text{ is the encoding of sentence A} \\ \mathbf{B} \text{ is the encoding of sentence B} \\ \|\mathbf{A}\|, \|\mathbf{B}\| \text{ are the norms of the vectors} \end{cases} \tag{3.5}$$

Having both semantic similarity and entity similarity scores, we can define an arbitrary threshold to accept an answer as correct. To determine this threshold, we introduce two new variables: ***PES*** and ***PSS***, which represent the entity similarity and semantic similarity between the paraphrased answer and the correct answer, respectively, and an arbitrary margin of error ***E*** to accommodate deviations in semantics, chosen words, and minor mistakes.

Using the formulas 3.6 and 3.7, where ***SES*** and ***SSS*** represent the student's entity and semantic similarity compared to the reference answer, we can determine whether the entity score and the semantic score are deemed sufficient for an answer.

$$EntityScorePassed = Sign(SES * (100 - E) - PES) \tag{3.6}$$

$$SemanticScorePassed = Sign(SSS * (100 - E) - SSS) \tag{3.7}$$

The user's responses are considered acceptable if the results of both equations 3.6 and 3.7 are 1; if not, then the responses are deemed unsatisfactory.

3.3 Multimodal Analysis Component

The Multimodal Analysis Component in our system is responsible for verifying confidence and detecting uncertainty in the responses given by the students. It is an integral part of making our evaluation pipeline more human-like. The idea is that in any oral examination conducted by humans, it is not enough to evaluate only the content of the responses; extra attention must be paid to "how" this content is delivered. An examiner, whether consciously or unconsciously, detects uncertainty in the delivery of an answer by a student and grades them accordingly. Our work focuses on detecting uncertainty using prosodic features (e.g., voice frequency, total time of silence, etc.), which we discuss thoroughly in Section 3.3.1. To reinforce our results from analyzing prosodic features, we introduce another method of evaluating uncertainty using facial features (e.g., direction of gaze and displayed emotion), described in Section 3.3.2.

3.3.1 Prosodic based evaluation

To measure uncertainty from the prosodic features of speech, three classifiers were trained using different algorithms. Our prosodic evaluation system builds upon the work of Pon-Barry [39], who conducted an examination to find the correlation between prosodic features and uncertainty in spoken language. They recorded speeches from individuals under different levels of uncertainty, including self-ratings and perceived ratings, and then analyzed the speech to determine how prosodic features (e.g., pitch, volume, timing) are linked to the degree of confidence in the content of the speech. The study found that perceived certainty tends to rate speeches as more certain compared to the self-ratings of the speaker. According to their findings, the most significant variables from the extracted prosodic features that explain the correlation are

temporal features (e.g., total silence time, total duration of speech, speaking duration). In Table 3.2, the correlations between prosodic features and mean perceived ratings are shown. For our case, we are only interested in the features that explain uncertainty in the entire utterance. Expanding on the findings of the paper and the data produced through their examinations,

Feature	Results from paper
min $f0$	0.074
max $f0$	-0.102
mean $f0$	-0.039
stdev $f0$	-0.078
range $f0$	-0.136^{*}
rel. position min $f0$	0.002
rel. position max $f0$	0.073
absolute slope $f0$	0.312^{**}
absolute slope 2 $f0$	Not Mentioned
min RMS	0.085
max RMS	-0.076
mean RMS	0.008
stdev RMS	-0.015
rel. position min RMS	0.039
rel. position max RMS	-0.085
total silence	-0.644^{**}
percent silence	-0.459^{**}
total duration	-0.653^{**}
speaking duration	-0.515^{**}
speaking rate	0.134^{*}

Table 3.2: Feature Analysis of Whole Utterance

we experimented by selecting features and algorithms to train classifiers for the task. From the features present in the data, we chose the following:

- ***PercentSilence***: The percentage of silence with respect to the total length of the speech.
- ***min*** $f0$: The minimum fundamental frequency ($f0$) observed in a speech segment.
- ***max*** $f0$: The maximum fundamental frequency.
- ***mean*** $f0$: The average fundamental frequency over the entire speech.
- ***stdev*** $f0$: The standard deviation of the fundamental pitch.

- ***range*$f0$**: The difference between the maximum and minimum fundamental frequencies.
- ***relposmin*$f0$**: The relative position within the speech segment where the minimum $f0$ occurs.
- ***relposmax*$f0$**: The relative position within the speech segment where the maximum $f0$ occurs.
- ***absslope1***: The absolute slope of the first segment of the $f0$ contour, reflecting how quickly the pitch changes at the beginning of the speech segment.
- ***absslope2***: The absolute slope of the second segment of the $f0$ contour, representing the rate of pitch change later in the speech segment.
- ***spkRate***: The speaking rate, measured in terms of words per second.
- ***Gender***: The gender of the speaker.

The reason we did not include *TotalDuration* and *SpeakingDuration* is that they are technically explained by other variables and, since they are unbounded, they are difficult to standardize.

To convert this task from a regression problem into a classification problem, any average perceived rating over 3 is considered as confident speech, while ratings below 3 are classified as uncertain and unconfident.

Training a random forest-based classifier proved effective, achieving an accuracy of 74% and a recall of 81% over the confident category. The performance metrics, including accuracy, precision, and other measures, are shown in Table 3.3.

	Precision	**Recall**	**F1-Score**
Uncertain	0.64	0.59	0.62
Confident	0.78	0.81	0.80
Accuracy	0.74 (150 samples)		
Macro Avg	0.71	0.70	0.71
Weighted Avg	0.73	0.73	0.73

Table 3.3: Performance metrics for the random forest based classifier.

Further improvement was achieved with gradient boosting methods, specifically XGBoost, which reached an accuracy of 70%. This method also showed lower recall, precision, and F1-score for both classes. The performance metrics for gradient boosting are displayed in Table 3.4.

	Precision	Recall	F1-Score
Uncertain	0.59	0.56	0.57
Confident	0.76	0.78	0.77
Accuracy	0.70 (150 samples)		
Macro Avg	0.67	0.67	0.67
Weighted Avg	0.70	0.70	0.70

Table 3.4: Performance metrics for the XGboost based classifier.

The best results were obtained using a neural network-based classifier, which achieved an accuracy of 77% and a recall of 97% over the confident category. The complete metrics data are presented in Table 3.5.

	Precision	Recall	F1-Score
Uncertain	0.88	0.41	0.56
Confident	0.74	0.97	0.84
Accuracy	0.77 (150 samples)		
Macro Avg	0.81	0.69	0.70
Weighted Avg	0.79	0.77	0.74

Table 3.5: Performance metrics for the Neural Network based classifier.

As seen in our results, our approach accepts a reduction in precision for uncertain cases to achieve improved precision and recall for confident speech instances. This is because false negatives —instances where confident individuals are incorrectly classified as uncertain— adversely impact user experience. Therefore, we aim to be cautious in our classification of uncertain cases. The loss of accuracy and recall is compensated for when the prosodic evaluation is used together with facial feature evaluation, reducing false negatives to a minimum while maintaining or improving overall accuracy.

3.3.2 Facial features based evaluation

To analyze the facial features from the video footage, we focused on gaze direction and expressed emotions. This section is primarily based on the findings of "The Look of (Un)confidence: Visual Markers for Inferring Speaker Confidence in Speech" [40]. The study suggests that observers can perceive a speaker's static confidence level solely by assessing visual cues. However, while evidence shows that this is possible, the question of "which visual cues are related to confidence level" remains unanswered. The mentioned study attempts to identify these visual

cues by conducting interviews and asking questions from a group of people, gathering both observers' perceptions of speakers' confidence and the speakers' own perceptions of their confidence. It was shown that observers accurately perceived confidence levels by using visual cues such as head movements, eye level, and facial expressions of the speaker. In their study, they collected 249 videos from which a group of six extracted the visual cues. Overall, there were 123 instances of low-confidence responses and 126 instances of high-confidence responses. Table 3.6 shows the proportion of visual cues associated with each confidence class.

Visual Cue	High Confidence (%)	Low Confidence (%)
Sustained Eye Contact	20.8	8.6
Upward Gaze	3.0	17.1*
Downward Gaze	14.9	23.8*
Sideways Gaze	10.9	21.9*
Serious Expression	51.5	9.5
Thinking Expression	3.0	13.3*
Still Posture	66.3	48.6

Table 3.6: Percentage of Items with Specific Visual Cues for High and Low Confidence Conditions

Now, using Bayes' theorem (Equation 3.8), along with the data provided in the study and utilizing Equation 3.9, it is possible to calculate the probability of having low confidence when encountering each visual cue.

$$P(\text{LC}|\text{Visual Cue}) = \frac{P(\text{Visual Cue}|\text{LC}) \cdot P(\text{LC})}{P(\text{Visual Cue})} \tag{3.8}$$

$$P(\text{Visual Cue}) = P(\text{Visual Cue}|\text{HC}) \cdot P(\text{HC}) + P(\text{Visual Cue}|\text{LC}) \cdot P(\text{LC}) \tag{3.9}$$

Table 3.7 shows the probabilities of having low confidence given each visual cue.

Visual Cue	P(Low Confidence | Visual Cue)
Sustained Eye Contact	≈ 0.291
Upward Gaze	≈ 0.848*
Downward Gaze	≈ 0.696*
Sideways Gaze	≈ 0.815*
Serious Expression	≈ 0.149
Thinking Expression	≈ 0.686*
Still Posture	≈ 0.449

Table 3.7: Probability of Low Confidence Given a Specific Visual Cue

Since we are only interested in cues that indicate low confidence, we discard those cues that are correlated with high confidence and keep only those marked in the tables.

Given that these visual cues are not completely independent, we approximate the probability of being unconfident given multiple visual cues using the inclusion-exclusion formula (Equation 3.10), treating them as independent variables.

$$P(\text{LC} \mid \text{visualcues}) = 1 - \prod_{i=1}^{n} (1 - P(\text{LC} \mid \text{visualcue}_i)) \tag{3.10}$$

By using Equation 3.10 and detecting facial features and visual cues, the facial features analysis component outputs the probability of being unconfident. This probability can be used along with the prosodic-based evaluation to measure the likelihood of low confidence in the given answer without any prior data about the validity of the response.

4
Implementation Setup

In this chapter, the real-world implementations of the concepts introduced in Chapter 3 are discussed in depth. First, in Section 4.1, the technologies and tools used to set up the development environment are briefly described. In Section 4.2, each internal module of the project is dissected and demonstrated thoroughly. Input handling is explained in Section 4.2.1. Question generation is analyzed in Section 4.2.2, where the LLMs used and the implementation details are described. Answer assessment is discussed in Section 4.2.3, covering the usage of language models and how they assess the answers. Finally, Section 4.3 details the application's frontend and how it connects the different modules of the backend.

4.1 Tools Used

The scope of the project required the application to use multiple technologies and tools to function. Primarily, Python was used as the language of choice for powering the logic and the backend of the application, while JavaScript was chosen for the frontend. In the list below, each tool and its function are described in detail:

- **FastAPI[43]**: A modern Python web framework used for developing the RESTful API of the application's backend. Known for its high performance and support for asynchronous programming, FastAPI was selected for its ease of integration and rapid deployment capabilities, making it well-suited for our web application, which requires quick concept testing.

- **ReactJS[44]**: Chosen for the front-end due to its fast rendering and component-based architecture. React enables the development of single-page applications with a dynamic user interface capable of handling states and responding to client inputs in real-time.
- **Transformers[45] [46]**: A library developed mainly by Hugging Face with community contributions. It is crucial for implementing pre-trained language models used for various NLP tasks in the application, such as question generation and answer evaluation. The library provides access to state-of-the-art transformer models, which are integral to the application's core functionality.
- **SentenceTransformers (SBERT)[47]**: Built on top of the Transformers library, SBERT supports a wide range of applications, including semantic search, semantic textual similarity, and paraphrase mining. It was extensively used in our project to derive sentence embeddings for similarity measurement, allowing for a deeper understanding of semantic similarity beyond simple keyword matching.
- **Llama 3[8]**: A series of LLMs developed and maintained by Facebook. This pre-trained model was integrated into the system for question generation and answer assessment, enabling the generation of questions based on content extracted from user-provided documents and evaluating the relevance and correctness of answers against expected responses.
- **scispaCy[19]**: A tailored version of the spaCy library designed for scientific and biomedical text processing. It was integrated into the application to handle domain-specific terminology and ensure accurate parsing of complex scientific language.
- **Whisper[48]**: An automated speech recognition (ASR) model developed by OpenAI. Whisper was used to transcribe audio input with high precision, enabling the conversion of speech to text and facilitating textual analysis of the input.
- **OpenCV[49]**: An open-source computer vision library used for processing video inputs. OpenCV was employed to identify visual cues in responses, such as facial expressions and eye movements, contributing to a more accurate assessment of user confidence and certainty.

4.2 Internal Modules

To implement the theories introduced in Chapter 3 using the tools discussed in Section 4.1, different modules were created for each stage of the application. Some of these modules cor-

respond to the theoretical components described earlier: input handling (Section 4.2.1), question generation (Section 4.2.2), and answer assessment (Section 4.2.3) collectively form the natural language component. Meanwhile, visual certainty assessment (Section 4.2.4) and auditory certainty assessment (Section 4.2.5) constitute the multimodal component.

4.2.1 Input Handling

Input handling manages the processing and collection of all files uploaded to the application, including text files for initiating the exam creation process and video recordings of the participant at the end of an exam for evaluation.

PDF Handling

Upon a POST request to `/uploadPDF`, a UUID is generated to serve as the session token, and the uploaded PDF file is saved as `{uuid}/raw.pdf` on the server. After a successful upload, the digestion process begins. This process extracts useful text from the file, identifies and separates each chapter, and finally divides the resulting chapters into smaller texts with overlapping tokens of an arbitrary length (1500 in our application), depending on the input length constraints of the LLM used in later stages.

Since well-formed PDFs do not contain data about chapters and sections, our approach involves parsing the PDF files statistically. First, we count all occurrences of fonts and their sizes. We assume that the most frequently used font size corresponds to the main text of the document.

To identify chapters and sections, we use regular expressions to find font sizes associated with the term "chapter" (`.*chapter.*`) or those that include a digit followed by the name of a chapter. We then save the font sizes associated with these occurrences and select the largest font size as the chapter size.

If regular expressions fail to identify the correct size, the entire text is divided into multiple sections based on all the font sizes used. We calculate the variance of the length of each section and the average coverage they provide over the whole text. Additionally, if the document is divided into fewer than an arbitrary number of sections (e.g., 7 sections), this division is deemed unacceptable. The variance and interquartile range are used to exclude results that produce too many small sections. Ultimately, the largest font size determines the start of a new chapter based on its frequency of occurrence.

After recognizing the chapters, the text is divided into smaller sections. Since these sections might not precisely incorporate the main text, we calculate the similarity between the sections by applying TF-IDF to the texts. We then determine where the first main chapter starts and ends, which helps in discarding sections such as references and introductions. This process ensures that the text is refined and ready for review by the user.

Video Handling

While answering the questions, each time the participant clicks the "Next" button to view the subsequent question, the timestamp is recorded. This timestamp, along with the video recording, is sent to our server via a POST request to `/processVideo/uuid`, where `uuid` is the session token for the user. The video is then split into smaller segments using the timestamps and the FFmpeg library [50][51]. Each segment is processed by the Whisper speech-to-text conversion model, resulting in transcriptions used for answer assessment. The segmented videos serve as raw data for certainty assessment.

4.2.2 Question Generation

To generate questions, the user first specifies the sections of the text from which they want questions and sends this information to the backend via a POST request to `/createtheexam/uuid`. As outlined in section 4.2.2, overlapping text segments are used to augment a prompt, which is then utilized during the inference of an LLM, specifically LLama 2 in our implementation. The LLM assesses whether the text contains sufficient information to create in-depth questions and verifies if the content pertains to the medical sector. If these conditions are not met, the generation process is bypassed to prevent the inclusion of low-quality questions in the test. Additionally, as discussed in section 3.2.1, to carefully select the generated questions, the measure in equation 3.7 requires identifying entities within each sentence using scispaCy's "en_core_sci_lg" pretrained model, tailored for medical and scientific contexts. For the measure in equation 3.6, Microsoft's "mpnet-base" model [52] is used within the Sentence Transformers library.

Since the generation of questions takes a considerable amount of time, we demonstrate in our testing in chapter 5 that by introducing a delay between the request for exam creation and the participant's response to the first question, the process can effectively be conducted in real-time.

At the end of a successful question generation, a TSV file will be created, which contains the

question, a reference answer, an answer without any unnecessary information, a paraphrased answer, and a completely wrong answer.

4.2.3 Answer Assessment

As explained in section 3.2.3, the response received from the participant undergoes a substantial process in which the transcription derived from the submitted video (section 4.2.1) is cleaned of words that are unnecessary for evaluation and is augmented with additional data to assist with the evaluation. To obtain the lemma of each word in a sentence, NLTK's [53] Porter Stemmer [54] is used, and to standardize numerics, the inflect [55] library is used to convert them into words, ensuring consistency between the original material and the results from speech-to-text.

Continuing with the concepts introduced in the mentioned section, we use scispaCy and Sentence Transformers exactly as described in section 4.2.2 to calculate similarity scores. However, to compare entities, we use a partial matching process in which two entities are considered equal if their Ratcliff/Obershelp similarity score [56] (i.e., the longest common subsequence (LCS) between them relative to their whole length) is more than 0.90.

At the end of the answer assessment, a list of scores the student has received, along with a simple flag for each question indicating whether the answer was sufficient or not, will be saved on the server to be served later to the participant.

4.2.4 Visual Certainty Assessment

Given the information presented in section 3.3.2, to assess certainty, we only need to obtain the visual cues that occur in a given video and solve the equation 3.10. To examine the video frame by frame, we use OpenCV [49]. Each frame is processed using the "shape predictor 68 landmarks" [57] [58], and the landmarks of the face are derived. Using these landmarks, the gaze direction is obtained, and with the FER (Face Emotion Recognition) library [59], the emotion of the face is inferred.

To avoid misidentifying cues, we only consider a cue to be visible if it occurs more than twice per the video's framerate, which is equivalent to two seconds. This approach helps in avoiding the recognition of cues that might be introduced by accident. The results of the assessment of each response will be appended to the grades from the previous section and served later to the participant.

4.2.5 Auditory Certainty Assessment

To use the developed neural network-based model from section 3.3.1, we employ the library *librosa* [60] to extract auditory features such as minimum and maximum fundamental frequencies, speaking rate, etc. To accurately determine the percentage of silence in speech, we use Whisper's transcription to identify the start and end segments of the response. This is important because a portion of the student's time may be spent reading the presented question in silence, and these segments should not be considered as input for evaluation.

4.3 Integration (UI)

The UI, which acts as a bridge for all the components, was developed entirely in ReactJS as a single-page application to demonstrate the feasibility of such an application and facilitate the testing of the developed features.

The application was designed with a straightforward, minimalist concept to avoid influencing or interfering with the participants' exam experience. When opened for the first time, as shown in figure 4.1, the participant can enter their email to receive the results later and upload a PDF file to initiate the exam creation process. For development purposes, there is also a field to enter the backend server's address for debugging any issues that may arise during testing.

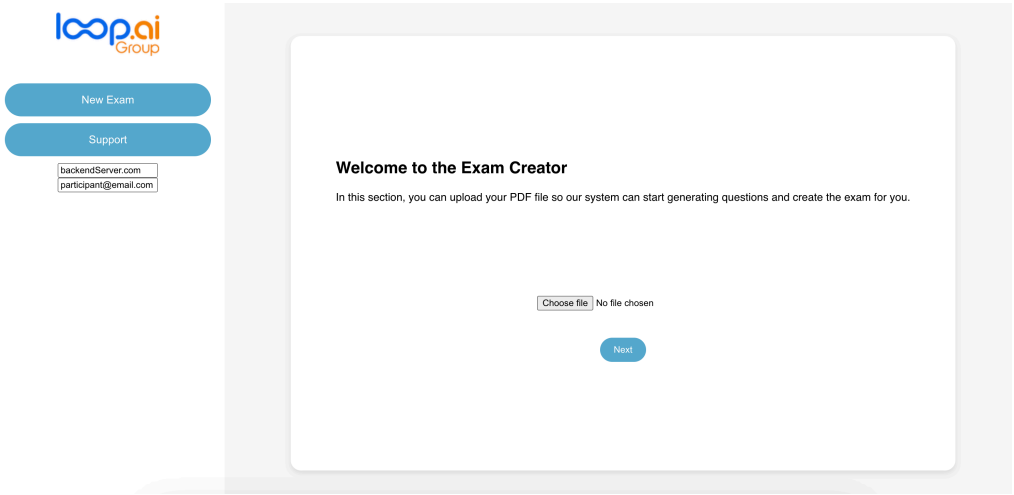


Figure 4.1: The first page of the application.

After a successful upload, the user will need to wait for a short period of time (figure: 4.2), during which the system will execute the text digestion process. If there are any issues with the

input data (e.g., text that is illegible or incompatible with the system's parameters), an error notification will be sent to the user. Conversely, if the input handling process is successful, the user will be redirected to the next page. Upon successful digestion, the user will be prompted

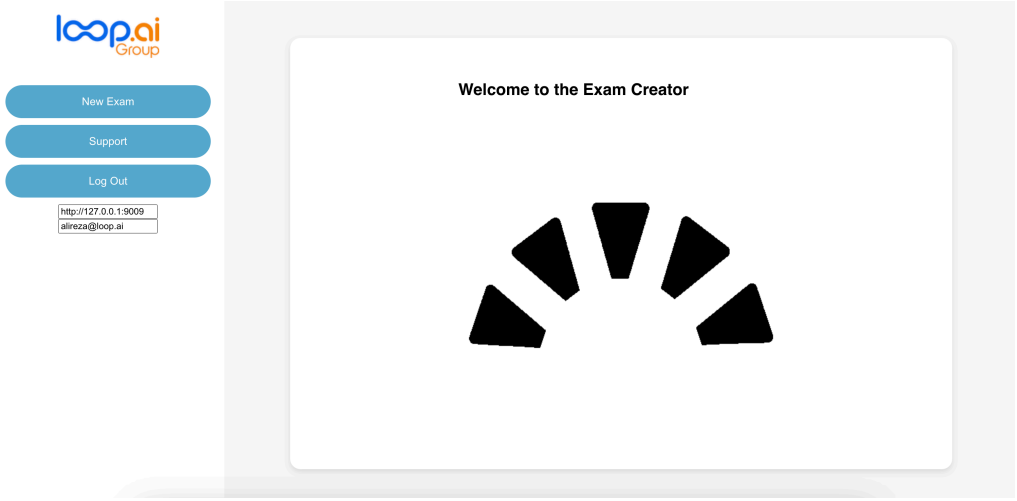


Figure 4.2: The application synthesising the uploaded PDF.

to review the results of the sectionalized input, as shown in figure 4.3. On this page, the user can select which sections to include in the question generation process by enabling or disabling the checkboxes next to each body of text. This allows the user to focus on specific parts of the book for practice. Additionally, if there are issues with identifying the main sections of the text, the user has the option to correct these errors on this page.

Upon requesting question generation after reviewing the material on the previous page, the user is directed to a new page where they must wait for 5 minutes. During this time, the user will read the regulations while the exam is being prepared. The user must grant access to their microphone and camera to be recorded, and a live feed of the user's recording is displayed in the top left frame of the application, allowing them to ensure they are being recorded properly as shown in figure 4.4.

Once the exam starts, the user will be presented with a question. They must answer it while remaining in the camera frame and maintaining an audible volume. After completing each response, they should click "next question" to proceed to the next question as shown in figure 4.5.

After answering all the questions, the participant has the option to submit their video response. The submission duration may vary depending on the participant's internet speed and camera quality, which could lead to longer waiting times. After a successful submission, par-

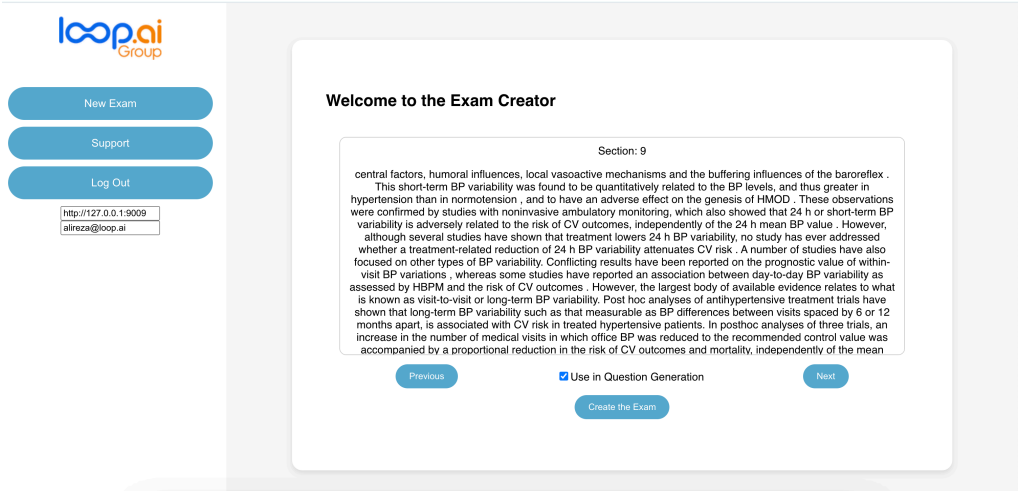


Figure 4.3: Application in review mode, the user has chosen to keep this section

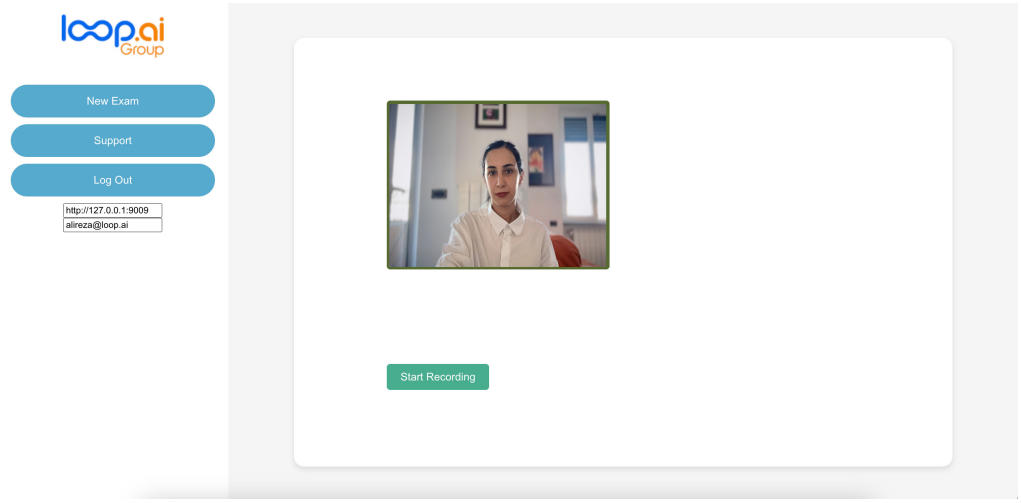


Figure 4.4: A student ready to take the exam inside application

ticipants will receive a notification that they can expect an email with their results, sent to the specified email address (figure 4.6.

Depending on the length of the submitted video, participants will receive an email with access to their evaluation results once the evaluation process is complete, as shown in figure 4.7.

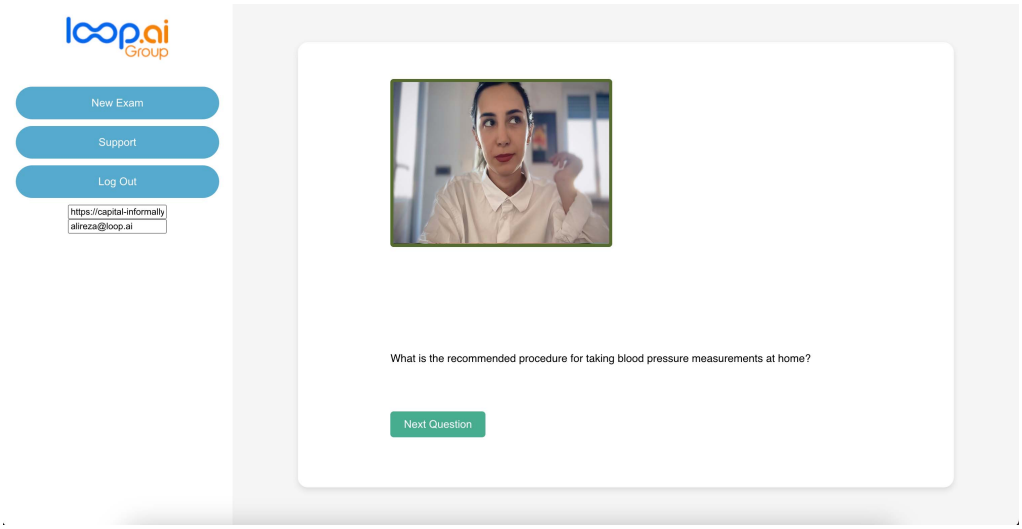


Figure 4.5: A student is taking the exam inside application

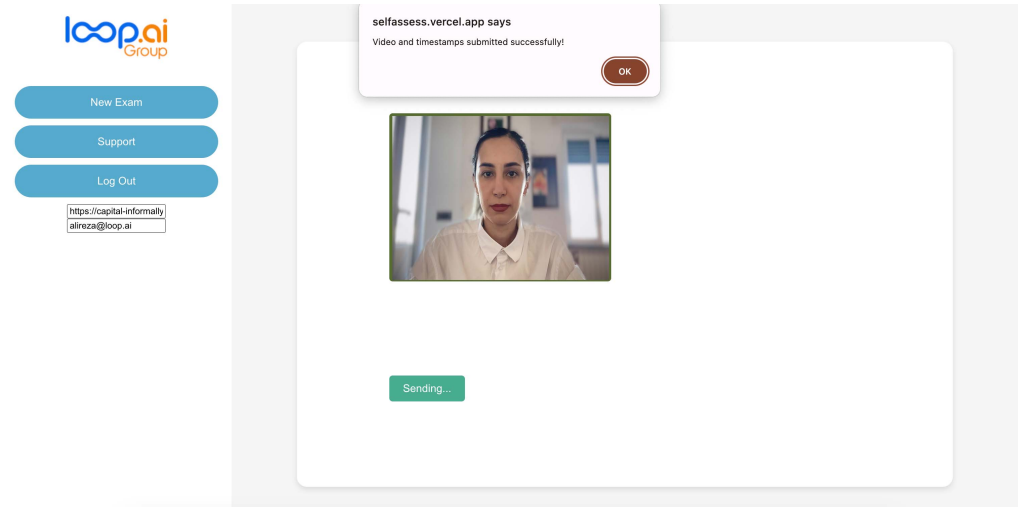


Figure 4.6: A student has finished the exam and has sent their responses

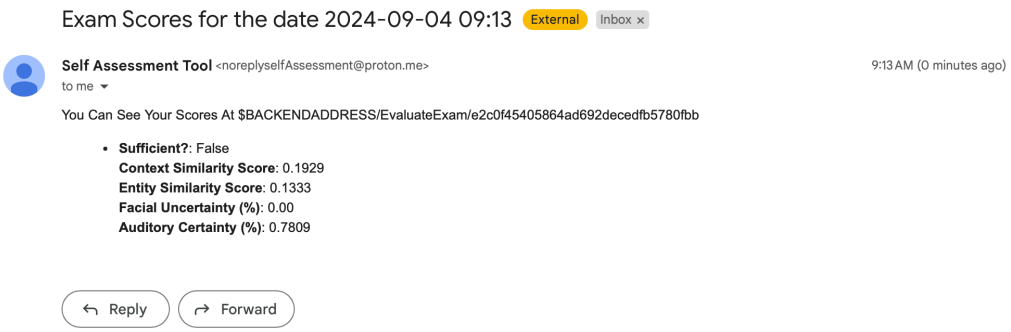


Figure 4.7: The scores of a taken text emailed to the student

5 Discussion

In this chapter, we delve into the experimental results obtained from our study, providing a comprehensive interpretation and analysis. We will examine the methodologies employed during the testing phase, evaluate the performance of our system, and discuss the outcomes in the context of our research objectives. Additionally, we will explore the challenges and limitations encountered throughout the study.

5.1 Testing Methodology

To assess the quality of the question generation and answer evaluation components of our system, we constructed a dataset using our question generation pipeline. The dataset was organized into six columns:

1. **Question**: The generated question.
2. **Reference Answer**: The ideal answer based on the source material.
3. **Perfect Answer**: A completely accurate response that matches the reference answer.
4. **Paraphrased Answer**: A correct response expressed in different wording or structure.
5. **Fair Answer**: A mostly correct response with minor inaccuracies or omissions.
6. **Wrong Answer**: An incorrect response demonstrating a misunderstanding of the material.

To obtain an accurate estimate of our system's performance, we needed to specify a minimum sample size. Using the formula 5.1 and knowing that our evaluation scale consists of discrete values ranging from 1 to 5, we assumed the worst-case scenario, where the variance is at its maximum. For a margin of error of 0.5, and defining the variance as 2, we calculated that at least 31 samples are required to draw conclusions about a variable with a 95% confidence level. For this experiment we were able to produce a sample size of 85.

$$\text{Sample Size} = \left(\frac{Z \cdot \sigma}{E}\right)^2 \tag{5.1}$$

From our dataset, we randomly selected questions to create a representative sample. For each question, one of the five possible answers was randomly chosen, resulting in a secondary dataset of question-answer pairs.

An expert from Mohammed Premier University, Intissar Haddiya, was then invited to evaluate the dataset. The expert assessed the quality and appropriateness of each question on a scale of 1 to 5, where a rating of 1 indicated the question was inappropriate and should not be asked, 2 meant it was incorrect, 3 signified it was wrongly stated or not fully correct, 4 implied it was interesting but not relevant, and 5 denoted the question was correct and well stated. These ratings helped determine the suitability of each question for assessing a student's understanding in an oral exam. Additionally, the expert rated the answers on a scale of 1 to 5 based on their validity and quality, without prior knowledge of the type of answer (reference, perfect, paraphrased, fair, or wrong). A rating of 1 indicated the answer was inappropriate and should not be provided due to being totally wrong, 2 signified the answer was incorrect, 3 meant the answer was correct but strongly incomplete, 4 indicated it was correct but partially complete, and 5 denoted the answer was both correct and totally complete. A sample of the question-answer pairs evaluated by the expert can be seen in table 5.1.

Question	**Proposed Answer**	**Question Grade**	**Answer Grade**	**Notes**

What interventions are effective in improving medication adherence and blood pressure control in hypertensive patients, as mentioned in the text?	The text mentions goal setting, provision of feedback, self-monitoring, follow-up visits, motivational interviewing, and promotion of self-sufficiency as interventions that have been proven effective in improving medication adherence and blood pressure control in hypertensive patients.	5	5	
What steps should be taken to diagnose and manage hypertension in children and adolescents according to the text?	The text suggests that children as young as three years old should be diagnosed with hypertension based on a single reading of their blood pressure, even if it's above the 95th percentile.	5	1	
What noninvasive and reproducible measure of renal and systemic arterial compliance or resistance can help evaluate renal organ damage and has a prognostic role for CV morbidity, mortality and renal outcomes in essential hypertensive patients, in CKD and in various CV diseases, besides albuminuria and eGFR?	The pulse rate is a noninvasive and reproducible measure of renal and systemic arterial compliance or resistance that can help evaluate renal organ damage and has a prognostic role for CV morbidity, mortality and renal outcomes in essential hypertensive patients, in CKD and in various CV diseases, besides albuminuria and eGFR.	5	1	

In the context of acute intracranial hemorrhage, what moderate BP reduction is recommended based on recent evidence, and what cautious BP reduction should be considered for patients with acute intracerebral hemorrhage and an SBP ≥220 mmHg?	A moderate BP reduction of 12.1 mmHg and a cautious BP reduction to 180 mmHg are recommended in the context of acute intracranial hemorrhage based on recent evidence.	5	1	The BP value is wrong

Table 5.1: A sample of the evaluated questions by the expert.

This evaluation served multiple purposes:

1. To determine if our question generation pipeline can find a generate a worthy question given a body of text.
2. To see if our system identifies the corresponding correct answer for a question from a body of text.
3. To assess the effectiveness of our evaluation methods in comparison to the expert ratings.
4. To establish the degree of deviation from the reference answer that can still be considered a valid response from an expert's perspective.

5.2 Results and Comparisons

In this section, we present the results of our experiments, focusing on the performance of our system based on insights provided by the human expert involved in the evaluation process.

Our expert's evaluation showed that the question generation pipeline worked impressively well, with 84% of the questions being counted as great questions with a rating of $5/5$, while

only 5% of the questions were deemed hard or not of quality, with no bad questions that are not eligible to be asked in an oral exam session, given our sample size, the mean question quality score is $4.79 \pm 2.4\%$ with a 95% confidence level. In the table 5.2, the results of the human evaluation are shown.

Expert evaluation	Questions	Correct Answers	Partial Answers	False Answers
5	84%	66%	75%	16%
4	12%	19%	7%	0%
3	5%	3%	0%	4%
2	0%	6%	7%	40%
1	0%	6%	11%	40%

Table 5.2: Statistics from the evaluations done by the expert, 1 is considered bad in quality and 5 means high quality

For our evaluation pipeline, we observed that 85% of the answers marked as correct received a score above 4, while incorrect answers were appropriately assigned lower scores, with 80% evaluated at or below 2. This indicates that, with a 95% confidence level, the true mean quality score is estimated to lie between 3.69 and 4.47 when our system identifies an answer to a question. This demonstrates the system's effectiveness in detecting correct answers from the input text.

To assess the effectiveness of our evaluation methods in relation to the expert's evaluations, we classified scores of 3, 4, and 5 as correct and scores of 1 and 2 as incorrect responses. The results show acceptable performance, with an accuracy of 67% and an F1-score of 73%. The F1-score indicates that our evaluation system performs well in detecting both incorrect and correct answers, while producing few false positives and false negatives.

Metric	Value	95% CI
Accuracy	0.67	[0.57, 0.76]
Precision	0.68	[0.55, 0.79]
Recall	0.80	[0.66, 0.88]
F1-Score	0.73	[0.60, 0.84]

Table 5.3: Performance metrics of our system's evaluation with regards to the expert's, along with 95% confidence intervals calculated using Clopper-Pearson interval.

Another interesting insight is that almost 82% of the partial answers are considered correct answers. According to our metrics, this indicates that the expert is willing to allow an average semantic deviation of 23% and an average deviation of nearly 27% in terms of the mentioned entities from the valid answer, while still marking the response as correct.

Metrics	Semantic similarity	Entities similarity
Average Deviation	0.23	0.27
Std. Deviations	0.17	0.18

Table 5.4: Deviation Metrics for Semantic similarity and Entities similarity.

5.3 Challenges and Limitations

One of the significant challenges in replicating real-world oral testing was to accurately model the grading process employed by experts. Although we incorporated prosodic and facial uncertainty detection into our system to mimic expert evaluation, our approach still falls short of capturing the subtle and often unconscious factors that may influence a professor's grading. This limitation underscores the complexity of human assessment and the difficulty in fully automating such processes.

A particularly hard challenge was the scarcity of data for prosodic and facial evaluation, especially in the context of facial feature detection. Not only did we require access to experts for data annotation, but we also faced limitations in manpower for conducting unbiased examinations and training models. As a result, we had to rely on existing literature and adapt those findings to suit our needs.

Another area that warrants improvement is the speed of question generation. Currently, our system can generate approximately one question per minute, which leaves room for improvements in real-time testing scenarios. Enhancing the generation speed would likely require better hardware or the development of more efficient models in future iterations.

Lastly, our research was constrained by the lack of comprehensive expert feedback. Although we obtained valuable insights from an expert, a broader consensus would have provided a more robust and less biased validation of our system's performance.

6

Conclusions and Future Work

The discussion presented in chapter 5 highlighted both the strengths and limitations of our system. While we have demonstrated that our question generation and answer assessment system can effectively produce relevant and accurate questions and is fully usable in an academic setting, further improvements can be made to enhance its applicability in real-world educational contexts.

Future work could address the challenges mentioned in section 5.3 to further enhance the system. To make the system's ability to model expert grading better, incorporating advanced algorithms along with real-life grading data could further capture the unique characteristics of human assessment. Expanding the dataset for prosodic and facial evaluations is also critical; this can be achieved by seeking collaborations to collect more data and improve annotation processes. Additionally, enhancing question generation speed may necessitate optimizing existing models and investigating new hardware solutions.

A straightforward improvement would be to leverage more powerful language models to increase the quality of questions and answers. Utilizing a model with a longer context window could enable the generation of more complex questions that span several concepts, thereby enriching the assessment process.

Finally, gathering feedback from a broader range of experts will be essential for achieving a more comprehensive and unbiased validation of the system. Addressing these issues will be critical to significantly improving the system's performance and its applicability in real-world educational settings.

References

[1] X. Han, Z. Zhang, N. Ding, Y. Gu, X. Liu, Y. Huo, J. Qiu, Y. Yao, A. Zhang, L. Zhang, W. Han, M. Huang, Q. Jin, Y. Lan, Y. Liu, Z. Liu, Z. Lu, X. Qiu, R. Song, J. Tang, J.-R. Wen, J. Yuan, W. X. Zhao, and J. Zhu, "Pre-trained models: Past, present and future," *AI Open*, vol. 2, pp. 225–250, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2666651021000231

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS'17. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 6000–6010.

[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *North American Chapter of the Association for Computational Linguistics*, 2019. [Online]. Available: https://api.semanticscholar.org/CorpusID:52967399

[4] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," *OpenAI*, 2018.

[5] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 1, jan 2020.

[6] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "Llama: Open and efficient foundation language models," 2023.

[7] H. Touvron, L. Martin, K. R. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez,

M. Khabsa, I. M. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom, "Llama 2: Open foundation and fine-tuned chat models," *ArXiv*, vol. abs/2307.09288, 7 2023. [Online]. Available: https://www.semanticscholar.org/paper/104b0bb1da562d53cbda87aec79ef6a2827d191a

[8] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Roziere, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. C. Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. M. Smith, F. Radenovic, F. Zhang, G. Synnaeve, G. Lee, G. L. Anderson, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. A. Ibarra, I. Kloumann, I. Misra, I. Evtimov, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van der Linde, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnstun, J. Saxe, J. Jia, K. V. Alwala, K. Upasani, K. Plawiak, K. Li, K. Heafield, K. Stone, K. El-Arini, K. Iyer, K. Malik, K. Chiu, K. Bhalla, L. Rantala-Yeary, L. van der Maaten, L. Chen, L. Tan, L. Jenkins, L. Martin, L. Madaan, L. Malo, L. Blecher, L. Landzaat, L. de Oliveira, M. Muzzi, M. Pasupuleti, M. Singh, M. Paluri, M. Kardas, M. Oldham, M. Rita, M. Pavlova, M. Kambadur, M. Lewis, M. Si, M. K. Singh, M. Hassan, N. Goyal, N. Torabi, N. Bashlykov, N. Bogoychev, N. Chatterji, O. Duchenne, O. Çelebi, P. Alrassy, P. Zhang, P. Li, P. Vasic, P. Weng, P. Bhargava, P. Dubal, P. Krishnan, P. S. Koura, P. Xu, Q. He, Q. Dong, R. Srinivasan, R. Ganapathy, R. Calderer, R. S. Cabral, R. Stojnic, R. Raileanu, R. Girdhar, R. Patel, R. Sauvestre, R. Polidoro, R. Sumbaly, R. Taylor, R. Silva, R. Hou, R. Wang, S. Hosseini, S. Chennabasappa, S. Singh, S. Bell, S. S. Kim, S. Edunov, S. Nie, S. Narang, S. Raparthy, S. Shen, S. Wan, S. Bhosale, S. Zhang, S. Vandenhende, S. Batra, S. Whitman, S. Sootla, S. Collot,

S. Gururangan, S. Borodinsky, T. Herman, T. Fowler, T. Sheasha, T. Georgiou, T. Scialom, T. Speckbacher, T. Mihaylov, T. Xiao, U. Karn, V. Goswami, V. Gupta, V. Ramanathan, V. Kerkez, V. Gonguet, V. Do, V. Vogeti, V. Petrovic, W. Chu, W. Xiong, W. Fu, W. Meers, X. Martinet, X. Wang, X. E. Tan, X. Xie, X. Jia, X. Wang, Y. Goldschlag, Y. Gaur, Y. Babaei, Y. Wen, Y. Song, Y. Zhang, Y. Li, Y. Mao, Z. D. Coudert, Z. Yan, Z. Chen, Z. Papakipos, A. Singh, A. Grattafiori, A. Jain, A. Kelsey, A. Shajnfeld, A. Gangidi, A. Victoria, A. Goldstand, A. Menon, A. Sharma, A. Boesenberg, A. Vaughan, A. Baevski, A. Feinstein, A. Kallet, A. Sangani, A. Yunus, A. Lupu, A. Alvarado, A. Caples, A. Gu, A. Ho, A. Poulton, A. Ryan, A. Ramchandani, A. Franco, A. Saraf, A. Chowdhury, A. Gabriel, A. Bharambe, A. Eisenman, A. Yazdan, B. James, B. Maurer, B. Leonhardi, B. Huang, B. Loyd, B. D. Paola, B. Paranjape, B. Liu, B. Wu, B. Ni, B. Hancock, B. Wasti, B. Spence, B. Stojkovic, B. Gamido, B. Montalvo, C. Parker, C. Burton, C. Mejia, C. Wang, C. Kim, C. Zhou, C. Hu, C.-H. Chu, C. Cai, C. Tindal, C. Feichtenhofer, D. Civin, D. Beaty, D. Kreymer, D. Li, D. Wyatt, D. Adkins, D. Xu, D. Testuggine, D. David, D. Parikh, D. Liskovich, D. Foss, D. Wang, D. Le, D. Holland, E. Dowling, E. Jamil, E. Montgomery, E. Presani, E. Hahn, E. Wood, E. Brinkman, E. Arcaute, E. Dunbar, E. Smothers, F. Sun, F. Kreuk, F. Tian, F. Ozgenel, F. Caggioni, F. Guzmán, F. Kanayet, F. Seide, G. M. Florez, G. Schwarz, G. Badeer, G. Swee, G. Halpern, G. Thattai, G. Herman, G. Sizov, Guangyi, Zhang, G. Lakshminarayanan, H. Shojanazeri, H. Zou, H. Wang, H. Zha, H. Habeeb, H. Rudolph, H. Suk, H. Aspegren, H. Goldman, I. Damlaj, I. Molybog, I. Tufanov, I.-E. Veliche, I. Gat, J. Weissman, J. Geboski, J. Kohli, J. Asher, J.-B. Gaya, J. Marcus, J. Tang, J. Chan, J. Zhen, J. Reizenstein, J. Teboul, J. Zhong, J. Jin, J. Yang, J. Cummings, J. Carvill, J. Shepard, J. McPhie, J. Torres, J. Ginsburg, J. Wang, K. Wu, K. H. U, K. Saxena, K. Prasad, K. Khandelwal, K. Zand, K. Matosich, K. Veeraraghavan, K. Michelena, K. Li, K. Huang, K. Chawla, K. Lakhotia, K. Huang, L. Chen, L. Garg, L. A, L. Silva, L. Bell, L. Zhang, L. Guo, L. Yu, L. Moshkovich, L. Wehrstedt, M. Khabsa, M. Avalani, M. Bhatt, M. Tsimpoukelli, M. Mankus, M. Hasson, M. Lennie, M. Reso, M. Groshev, M. Naumov, M. Lathi, M. Keneally, M. L. Seltzer, M. Valko, M. Restrepo, M. Patel, M. Vyatskov, M. Samvelyan, M. Clark, M. Macey, M. Wang, M. J. Hermoso, M. Metanat, M. Rastegari, M. Bansal, N. Santhanam, N. Parks, N. White, N. Bawa, N. Singhal, N. Egebo, N. Usunier, N. P. Laptev, N. Dong, N. Zhang, N. Cheng, O. Chernoguz, O. Hart, O. Salpekar, O. Kalinli, P. Kent, P. Parekh, P. Saab, P. Balaji, P. Rittner, P. Bontrager, P. Roux, P. Dollar,

P. Zvyagina, P. Ratanchandani, P. Yuvraj, Q. Liang, R. Alao, R. Rodriguez, R. Ayub, R. Murthy, R. Nayani, R. Mitra, R. Li, R. Hogan, R. Battey, R. Wang, R. Maheswari, R. Howes, R. Rinott, S. J. Bondu, S. Datta, S. Chugh, S. Hunt, S. Dhillon, S. Sidorov, S. Pan, S. Verma, S. Yamamoto, S. Ramaswamy, S. Lindsay, S. Lindsay, S. Feng, S. Lin, S. C. Zha, S. Shankar, S. Zhang, S. Zhang, S. Wang, S. Agarwal, S. Sajuyigbe, S. Chintala, S. Max, S. Chen, S. Kehoe, S. Satterfield, S. Govindaprasad, S. Gupta, S. Cho, S. Virk, S. Subramanian, S. Choudhury, S. Goldman, T. Remez, T. Glaser, T. Best, T. Kohler, T. Robinson, T. Li, T. Zhang, T. Matthews, T. Chou, T. Shaked, V. Vontimitta, V. Ajayi, V. Montanez, V. Mohan, V. S. Kumar, V. Mangla, V. Albiero, V. Ionescu, V. Poenaru, V. T. Mihailescu, V. Ivanov, W. Li, W. Wang, W. Jiang, W. Bouaziz, W. Constable, X. Tang, X. Wang, X. Wu, X. Wang, X. Xia, X. Wu, X. Gao, Y. Chen, Y. Hu, Y. Jia, Y. Qi, Y. Li, Y. Zhang, Y. Zhang, Y. Adi, Y. Nam, Yu, Wang, Y. Hao, Y. Qian, Y. He, Z. Rait, Z. DeVito, Z. Rosnbrick, Z. Wen, Z. Yang, and Z. Zhao, "The llama 3 herd of models," 2024. [Online]. Available: https://arxiv.org/abs/2407.21783

[9] I. García-Ferrero, R. Agerri, A. Atutxa Salazar, E. Cabrio, I. de la Iglesia, A. Lavelli, B. Magnini, B. Molinet, J. Ramirez-Romero, G. Rigau, J. M. Villa-Gonzalez, S. Villata, and A. Zaninello, "MedMT5: An open-source multilingual text-to-text LLM for the medical domain," in *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, N. Calzolari, M.-Y. Kan, V. Hoste, A. Lenci, S. Sakti, and N. Xue, Eds. Torino, Italia: ELRA and ICCL, May 2024, pp. 11 165–11 177. [Online]. Available: https://aclanthology.org/2024.lrec-main.974

[10] A. Goel, A. Gueta, O. Gilon, C. Liu, S. Erell, L. H. Nguyen, X. Hao, B. Jaber, S. Reddy, R. Kartha, J. Steiner, I. Laish, and A. Feder, "Llms accelerate annotation for medical information extraction," in *Proceedings of the 3rd Machine Learning for Health Symposium*, ser. Proceedings of Machine Learning Research, S. Hegselmann, A. Parziale, D. Shanmugam, S. Tang, M. N. Asiedu, S. Chang, T. Hartvigsen, and H. Singh, Eds., vol. 225. PMLR, 10 Dec 2023, pp. 82–100. [Online]. Available: https://proceedings.mlr.press/v225/goel23a.html

[11] E. Chung, C. Choi, H. Gupta, D. Lee, J. Mullenbach, S. Singh, S. P, S. N, and P. Zhang, "Med-palm: A large language model for medical question answering," *arXiv preprint arXiv:2305.00394*, 2023. [Online]. Available: https://arxiv.org/abs/2305.00394

[12] S. Lee, P. Zhang, S. Singh, J. Mullenbach, H. Gupta, C. Choi, and E. Chung, "Med-palm 2: Advancing medical question answering with enhanced large language models," *arXiv preprint arXiv:2401.00743*, 2024. [Online]. Available: https://arxiv.org/abs/2401.00743

[13] N. Meißner, S. Speth, J. Kieslinger, and S. Becker, "Evalquiz – llm-based automated generation of self-assessment quizzes in software engineering education," in *Software Engineering im Unterricht der Hochschulen 2024*. Bonn: Gesellschaft für Informatik e.V., 2024, pp. 53–64.

[14] J. Kaddour, J. Harris, M. Mozes, H. Bradley, R. Raileanu, and R. McHardy, "Challenges and applications of large language models," *ArXiv*, vol. abs/2307.10169, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:259982665

[15] B. Chen, Z. Zhang, N. Langrené, and S. Zhu, "Unleashing the potential of prompt engineering in large language models: a comprehensive review," 10 2023.

[16] N. Knoth, A. Tolzin, A. Janson, and J. M. Leimeister, "Ai literacy and its implications for prompt engineering strategies," *Computers and Education: Artificial Intelligence*, vol. 6, p. 100225, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2666920X24000262

[17] T. F. Heston and C. Khun, "Prompt engineering in medical education," *International Medical Education*, vol. 2, no. 3, pp. 198–205, 2023. [Online]. Available: https://www.mdpi.com/2813-141X/2/3/19

[18] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, "spaCy: Industrial-strength Natural Language Processing in Python," 2020.

[19] M. Neumann, D. King, I. Beltagy, and W. Ammar, "ScispaCy: Fast and Robust Models for Biomedical Natural Language Processing," in *Proceedings of the 18th BioNLP Workshop and Shared Task*. Florence, Italy: Association for Computational Linguistics, Aug. 2019, pp. 319–327. [Online]. Available: https://www.aclweb.org/anthology/W19-5034

[20] J. H. Wolfe, "Automatic question generation from text - an aid to independent study," *SIGCSE Bull.*, vol. 8, no. 1, p. 104–112, feb 1976. [Online]. Available: https://doi.org/10.1145/952989.803459

[21] X. Yao, G. Bouma, Y. Zhang, P. Piwek, and K. Boyer, "Semantics-based question generation and implementation," *Dialogue Discourse*, vol. 3, 03 2012.

[22] M. Divate and A. Salgaonkar, "Automatic question generation approaches and evaluation techniques," *Current Science*, vol. 113, pp. 1683–1691, 11 2017.

[23] T. Alsubait, B. Parsia, and U. Sattler, "Ontology-based multiple choice question generation," *KI - Künstliche Intelligenz*, vol. 30, 11 2015.

[24] T. H. Tan, P. L. Teh, and Z. Yusoff, "Polyaqg framework: Auto-generating assessment questions," in *2021 IEEE International Conference on Computing (ICOCO)*, 2021, pp. 163–167.

[25] T. H. Tan and Z. Yusoff, "Polyaqg: An automated question generation framework based on a combination of four analytical approaches," *Mathematics and Computer Science*, vol. 7, no. 4, pp. 68–74, 2022. [Online]. Available: https://doi.org/10.11648/j.mcs.20220704.12

[26] M. Blšták and V. Rozinajová, "Automatic question generation based on sentence structure analysis using machine learning approach," *Natural Language Engineering*, vol. 28, no. 4, p. 487–517, 2022.

[27] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," in *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, J. Su, K. Duh, and X. Carreras, Eds. Austin, Texas: Association for Computational Linguistics, Nov. 2016, pp. 2383–2392. [Online]. Available: https://aclanthology.org/D16-1264

[28] A. Olney, "Generating multiple choice questions from a textbook: Llms match human performance on most metrics," *Workshop on Empowering Education with LLMs - the Next-Gen Interface and Content Generation at the AIED'23 Conference*, 2023. [Online]. Available: https://par.nsf.gov/biblio/10447593

[29] R. Nogueira, W. Yang, J. Lin, and K. Cho, "Document expansion by query prediction," *arXiv preprint arXiv:1904.08375*, 2019, submitted on 17 Apr 2019 (v1), last revised 25 Sep 2019 (v2). [Online]. Available: https://doi.org/10.48550/arXiv.1904.08375

[30] D. R. Cheriton, "From doc2query to doctttttquery," 2019. [Online]. Available: https://api.semanticscholar.org/CorpusID:208612557

[31] Microsoft, "Ms marco passage ranking leaderboard," 2024, accessed: August 29, 2024. [Online]. Available: https://microsoft.github.io/MSMARCO-Passage-Ranking-Submissions/leaderboard/

[32] S. M. Patil and S. Patil, "Evaluating student descriptive answers using natural language processing," *International Journal of Engineering Research & Technology (IJERT)*, vol. 3, no. 3, March 2014.

[33] A. Raut and N. Mahender, "Resolving single-sentence answers for the development of an automated assessment process," 04 2023, pp. 1–5.

[34] M. Rahman and F. Siddiqui, "Nlp-based automatic answer script evaluation," vol. 4, pp. 35–42, 12 2018.

[35] M. Agarwal, R. Kalia, V. Bahel, and A. Thomas, "Autoeval: A nlp approach for automatic test evaluation system," in *2021 IEEE 4th International Conference on Computing, Power and Communication Technologies (GUCON)*, 2021, pp. 1–6.

[36] M. F. Bashir, H. Arshad, A. R. Javed, N. Kryvinska, and S. S. Band, "Subjective answers evaluation using machine learning and natural language processing," *IEEE Access*, vol. 9, pp. 158 972–158 983, 2021.

[37] D. Kahneman and A. Tversky, "Variants of uncertainty," *Cognition*, vol. 11, no. 2, pp. 143–157, 1982. [Online]. Available: https://www.sciencedirect.com/science/article/pii/0010027782900233

[38] C. Dijkstra, E. Krahmer, and M. Swerts, "Manipulating uncertainty: The contribution of different audiovisual prosodic cues to the perception of confidence," *International Journal of Web Engineering and Technology - IJWET*, 01 2006.

[39] H. Pon-Barry, "Prosodic manifestations of confidence and uncertainty in spoken language," in *Interspeech*, 09 2008, pp. 74–77. [Online]. Available: https://api.semanticscholar.org/CorpusID:16791087

[40] Y. Mori and M. D. Pell, "The look of (un)confidence: Visual markers for inferring speaker confidence in speech," *Frontiers in Communication*, vol. 4, 2019. [Online]. Available: https://www.frontiersin.org/journals/communication/articles/10.3389/fcomm.2019.00063

[41] A. Emerson, P. Houghton, K. Chen, V. Basheerabad, R. Ubale, and C. W. Leong, "Predicting user confidence in video recordings with spatio-temporal multimodal analytics," in *Companion Publication of the 2022 International Conference on Multimodal Interaction*, ser. ICMI '22 Companion. New York, NY, USA: Association for Computing Machinery, 2022, p. 98–104. [Online]. Available: https://doi.org/10.1145/3536220.3558007

[42] South Eastern Sydney Local Health District. (2020, Nov.) Clinical abbreviations list. Accessed: 2024-04-21. [Online]. Available: https://www.seslhd.health.nsw.gov.au/sites/default/files/groups/Executive_Services/Clinical%20Abbreviations%20List%20-%20SESLHDPR%20282.pdf

[43] S. Ramírez, "Fastapi," Software library, 2024, accessed: 2024-08-21. [Online]. Available: https://fastapi.tiangolo.com

[44] React Contributors, "React," JavaScript library, 2024, accessed: 2024-08-21. [Online]. Available: https://reactjs.org

[45] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest, and A. Rush, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Q. Liu and D. Schlangen, Eds. Online: Association for Computational Linguistics, Oct. 2020, pp. 38–45. [Online]. Available: https://aclanthology.org/2020.emnlp-demos.6

[46] Hugging Face, "Transformers," Python library, 2024, accessed: 2024-08-21. [Online]. Available: https://github.com/huggingface/transformers

[47] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 11 2019. [Online]. Available: https://arxiv.org/abs/1908.10084

[48] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," 2022. [Online]. Available: https://arxiv.org/abs/2212.04356

[49] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[50] FFmpeg Developers, "Ffmpeg," Software, 2024, accessed: 2024-05-25. [Online]. Available: https://ffmpeg.org

[51] K. Kroening, "ffmpeg-python," GitHub repository, 2024, accessed: 2024-05-25. [Online]. Available: https://github.com/kkroening/ffmpeg-python

[52] Microsoft, "Mpnet base," Pre-trained model available on Hugging Face, 2024, accessed: 2024-08-21. [Online]. Available: https://huggingface.co/microsoft/mpnet-base

[53] S. Bird and E. Loper, "Natural language toolkit," Software package, 2024, accessed: 2024-08-21. [Online]. Available: https://www.nltk.org

[54] M. Porter, "An algorithm for suffix stripping," in *Program*, vol. 14, no. 3. Cambridge: Cambridge University Press, 1980, pp. 130–137, porter Stemmer algorithm.

[55] P. A. McMillan, "inflect: A library for generating plural, singular, and readable numbers," Python package, 2024, accessed: 2024-08-21. [Online]. Available: https://pypi.org/project/inflect/

[56] P. E. Black, "Ratcliff/obershelp pattern recognition," Dictionary of Algorithms and Data Structures [online], Paul E. Black, ed., January 2021, accessed: 2024-08-21. [Online]. Available: https://www.nist.gov/dads/HTML/ratcliffObershelp.html

[57] D. Developers, "Shape predictor 68 face landmarks model," Pre-trained model, 2024, accessed: 2024-08-21. [Online]. Available: http://dlib.net/files/shape_predictor_68_face_landmarks.dat.bz2

[58] V. Kazemi and J. Sullivan, "One millisecond face alignment with an ensemble of regression trees," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 1867–1874. [Online]. Available: https://doi.org/10.1109/CVPR.2014.241

[59] J. Shenk, "fer: Facial expression recognition," GitHub repository, 2024, accessed: 2024-08-21. [Online]. Available: https://github.com/JustinShenk/fer

[60] B. McFee, M. McVicar, D. Faronbi, I. Roman, M. Gover, S. Balke, S. Seyfarth, A. Malek, C. Raffel, V. Lostanlen, B. van Niekerk, D. Lee, F. Cwitkowitz, F. Zalkow, O. Nieto, D. Ellis, J. Mason, K. Lee, B. Steers, and W. Pimenta, "librosa/librosa: 0.10.2.post1 (0.10.2.post1)," 2024. [Online]. Available: https://doi.org/10.5281/zenodo.7746972

Acknowledgments

First and foremost, I would like to express my deepest gratitude to my supervisor, Professor Massimo Marchiori, for his guidance and support throughout this journey. His patience have been instrumental in shaping this thesis.

I would also like to extend my heartfelt thanks to Loop.AI for providing me with the opportunity to work on this project, and in particular to my supervisor at the company, Davide Baldelli. His consistent help, insightful feedback, and (endless) corrections of my mistakes have been crucial in creation of this system. Without his guidance, this thesis would have probably taken twice as long to complete-if not longer!

I would also like to extend my gratitude to Prof. Intissar Haddiya, PhD (University Mohammed First - Department of Nephrology, Faculty of Medicine and Pharmacy of Oujda, Morocco) for co-authoring with Loop AI the core concept behind our cognitive tool and for her expertise and support in evaluating the results of our system. Her deep medical background has been a fundamental pre-requirement for ensuring the accuracy and relevance of our work.

I am deeply grateful to my family for their support, understanding, and the uncountable mistimed FaceTimes during this journey. After finishing this I feel like I'm the greatest alive, thanks dad!

This work would not have been possible without the invaluable contributions and support of the individuals and groups. I sincerely appreciate their presence and their confidence in my abilities throughout this entire process.